

Problem families

QF-AgentOS — An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

collateral_allocation	1
payment_routing	2
settlement_netting	2
fraud_detection	2
rfq_fill	3
Adding a family	3

Five families, two task types. The pipeline is problem-agnostic: a family implements a `ProblemDomain` (optimization) or a `ClassificationDomain`, and plugs into the same agents, backends, verification, audit, and governance.

Family	Task type	Quantum technique	Verdict (<i>evidence</i>)
<code>collateral_allocation</code>	optimization	QAOA	classical preferred
<code>payment_routing</code>	optimization	QAOA	classical preferred
<code>settlement_netting</code>	optimization	QAOA	quantum parity
<code>fraud_detection</code>	classification	quantum kernel	classical preferred
<code>rfq_fill</code>	classification	quantum kernel	classical preferred

Read the “encoding loss” note in each section. For the optimisation families that is what actually decides the verdict — not the solver, not the hardware.

collateral_allocation

Post the cheapest set of securities that covers a margin requirement.

Objective minimise posting cost $\sum (\text{cost_bps}/1\text{e}4) \cdot \text{value} \cdot x$ · **s.t.** post-haircut coverage $\sum (1 - \text{haircut}) \cdot \text{value} \cdot x \geq \text{required_collateral}$, a minimum-HQLA floor, and per-issuer/counterparty concentration caps. $x \in \{0, 1\}$ (post the whole lot).

```
problem: collateral_allocation
constraints:
  required_collateral: 4000000
  minimum_hqla: 1000000
  concentration: { issuer: 0.40 }
inventory:
  - { id: BOND_A, issuer: UST, market_value: 2000000, haircut: 0.02, cost_bps: 5, hqla: true }
```

Classical LP lower bound + exact binary MILP (HiGHS). **Quantum** QUBO → Ising → QAOA on a reduced instance. **Encoding loss** Coverage is encoded, as a proper inequality via binary slack bits with an adaptive penalty. **Concentration and HQLA are dropped** (verification-only) — the QAOA optimum can breach them, and the verifier catches it. Example: `examples/collateral-allocation.yaml`.

payment_routing

Route each transaction to exactly one eligible route (a generalized assignment problem).

Objective minimise expected cost = processing + fixed fee + expected fraud + latency + expected decline $(1 - \text{approval}) \cdot \text{penalty}$ · s.t. one route per transaction, per-route capacity, network diversification, and a portfolio approval floor.

```
problem: payment_routing
routing: { decline_penalty_bps: 120, network_concentration: 0.60, min_overall_approval: 0.90 }
routes:   [{ id: R_VISA, cost_bps: 18, approval_rate: 0.94, capacity: 3, network: VISA }]
transactions: [{ id: T1, amount: 250000, eligible_routes: [R_VISA] }]
```

Classical sparse GAP MILP (HiGHS). **Quantum** one-hot assignment QUBO $\rightarrow$ QAOA. **Encoding loss** The QUBO encodes cost + the one-hot assignment penalty. **Capacity, network diversification, and the approval floor are dropped** (verification-only). Example: examples/payment-routing.yaml.

settlement_netting

RTGS gridlock resolution: pick a batch of queued interbank obligations to settle **simultaneously**, maximising settled value while nobody overdraws.

Objective minimise unsettled value (= maximise settled) · s.t. for each participant $\text{outflow}(\text{batch}) - \text{inflow}(\text{batch}) \leq \text{balance}$; optional settled-value floor.

```
problem: settlement_netting
settlement: { penalty_scale: 8.0 }
participants: [{ id: BANK_A, balance: 10 }, { id: BANK_D, balance: 60 }]
obligations: [{ id: O_AB, payer: BANK_A, payee: BANK_B, amount: 100 }]
```

The demo encodes a classic gridlock: $A \rightarrow B \rightarrow C \rightarrow A$ of 100 each, every bank holding only 10 — none can settle alone, but settling the cycle *simultaneously* nets to zero.

Classical exact MILP (HiGHS). **Quantum** QUBO with each participant's liquidity inequality **genuinely encoded** via binary slack bits (unlike the families above, which drop constraints).

! The structural finding

Liquidity is a **packing constraint**, and it does not survive QUBO encoding. Settling an obligation of amount a that overshoots a payer's headroom by δ gains reward $\sim a$ but costs only $A \cdot \delta^2$. Since δ can be arbitrarily small while a is large, **no tractable penalty A makes the ground state feasible** — you'd need $A \sim \text{max_amount} \cdot \text{total}$, which swamps the objective and destroys the QAOA landscape. A 320-instance adversarial sweep found 25–90% of QUBO ground states liquidity-infeasible; **every one was caught and reported honestly**. This is a property of the problem class, not a tuning failure.

Example: examples/settlement-netting.yaml.

fraud_detection

Binary classification on an imbalanced, temporally-ordered dataset.

Classical logistic regression (IRLS) + RBF kernel-ridge. **Quantum** ZZ feature-map **fidelity kernel** + the *same* kernel-ridge learner — so the comparison isolates the **kernel**, not the learner. **Method honesty** temporal holdout (no look-ahead), train-only standardisation, data-leakage checks, and a bootstrap significance test **against the RBF kernel** (not against the weakest classical model).

```
problem: fraud_detection
classification:
  target_metric: auc
  test_fraction: 0.3
  feature_budget: 4      # qubits = features fed to the quantum kernel
```

```
bootstrap: 500
synthetic: { n_samples: 240, n_features: 6, class_balance: 0.2 }
```

Inline features + labels (+ optional timestamps) are accepted instead of synthetic. **Encoding loss** The quantum kernel sees a *reduced* feature/sample instance (feature budget + an $O(n^2)$ sample cap) — not the full dataset the classical baselines see. Example: `examples/fraud-detection.yaml`.

rfq_fill

Will a request-for-quote fill? A market-maker problem: predict fill from features known at quote time — quoted spread, order size, volatility, response latency, counterparty tier.

Shares the entire `QuantumKernelClassificationDomain` pipeline with `fraud_detection`; only the synthetic generator differs (a $\text{spread} \times \text{size}$ **nonlinear** boundary — something a ZZ kernel could in principle exploit; the pipeline tests honestly whether it does. It doesn't: AUC 0.496, worse than a coin flip).

```
problem: rfq_fill
classification:
  target_metric: auc
  feature_budget: 4
  synthetic: { n_samples: 260, n_features: 6, class_balance: 0.35 }
```

Example: `examples/rfq-fill.yaml`.

Adding a family

1. Implement `ProblemDomain` (optimization) or subclass `QuantumKernelClassificationDomain` (classification) in `src/qf_agentos/finance/`.
2. Add its IR block + validation branch in `core/ir.py`.
3. Register it in `finance/__init__.py` (`KNOWN_PROBLEMS` + `get_domain`).
4. Add a `skill.yaml`, an example spec, and tests.

The optimization contract:

Method	Purpose
<code>requirements(spec)</code>	Understand + flag gaps
<code>formulations(spec)</code>	Enumerate LP/MILP/QUBO/Ising
<code>solve_classical_full(spec)</code>	The fair LP + MILP baseline
<code>reduce_to_instance(spec, max_qubits)</code>	Qubit-budgeted research instance
<code>build_qubo(instance, slack_bits)</code>	QUBO + documented encoding losses
<code>evaluate_bits(...)</code>	Decode bits → a verified <code>SolveResult</code>
<code>verify_full / verify_instance</code>	Exact constraint re-check
<code>instance_warm_start(...)</code>	Optional warm-start biases

Non-negotiable: `build_qubo` must declare its `encoding_losses`, and every decoded quantum solution must be re-checked against the exact constraints. That is what makes the verdict trustworthy. See [CONTRIBUTING.md](#).